

Reviewer Pack — Minimal Ramanujan Quotient and Resolution Proof Width Inequa...

Entry f3906e3e40c2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 14:29:33 UTC

Minimal Ramanujan Quotient and Resolution Proof Width Inequality

Entry ID: f3906e3e40c2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 14:29:33 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Ramanujan’s Q-function) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Boolean formula φ with n variables, the minimal value of Ramanujan’s Q-function $Q(\varphi)$ over all its possible evaluations is linearly correlated with its resolution proof width $w(\varphi)$, such that $Q_{\min}(\varphi) = \Theta(w(\varphi))$

Rationale (proposer’s reasoning):

Ramanujan’s Q-function provides a measure of ‘dispersion’ in number theory, which may mirror the complexity of the logical structure within a formula. A higher Q-value could correspond to a more intricate resolution proof, providing insight into the difficulty of resolving the formula.

Taxonomy category: ramanujan_q_function_resolution_width (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): beb724795d8bcd61

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For Boolean formulas φ with n variables, if the correlation coefficient from linear regression of $Q_{\min}(\varphi)$ vs. $w(\varphi)$ is ≥ 0.7 for all seeds, and the p-value < 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Ramanujan's Q-function" AND "resolution proof complexity" - "minimal Ramanujan quotient" AND "proof width inequality" - " $Q_{\min}(\phi) = \theta(w(\phi))$ "

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def generate_formula(n):
    if n == 1:
        return "x"
    else:
        left = generate_formula(random.randint(1, n-2))
        right = generate_formula(n - len(left) - 2)
        return f"({left} & {right})"

def calculate_ramanujan_q(formula):
    # Placeholder for Ramanujan's Q-function calculation
    # This is a dummy implementation and should be replaced with actual logic
    return random.random()

def calculate_resolution_width(formula):
    # Placeholder for resolution proof width calculation
    # This is a dummy implementation and should be replaced with actual logic
    return len(formula.split())

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        formula = generate_formula(n)
        q_value = calculate_ramanujan_q(formula)
        w_value = calculate_resolution_width(formula)

        results.append({
            "n": n,
            "q_value": q_value,
            "w_value": w_value
        })
```

```

    })

if not results:
    return {
        "metric_name": "Ramanujan Q and Resolution Width",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

q_values = [r["q_value"] for r in results]
w_values = [r["w_value"] for r in results]

n_max = max(r["n"] for r in results)
instances_tested = len(results)

# Placeholder for linear regression
def linear_regression(x, y):
    n = len(x)
    sum_x = sum(x)
    sum_y = sum(y)
    sum_xy = sum(xi * yi for xi, yi in zip(x, y))
    sum_xx = sum(xi ** 2 for xi in x)

    slope = (n * sum_xy - sum_x * sum_y) / (n * sum_xx - sum_x ** 2)
    intercept = (sum_y - slope * sum_x) / n
    return slope, intercept

slope, _ = linear_regression(q_values, w_values)

conjecture_holds = abs(slope) >= 0.7
counterexample = "" if conjecture_holds else "slope < 0.7"

return {
    "metric_name": "Ramanujan Q and Resolution Width",
    "metric_value": slope,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if "metric_value" not in trial_result or trial_result["metric_value"] is None:
            continue

```

```

        results.append(trial_result["metric_value"])

    if not results:
        print("RESULT: INCONCLUSIVE no_valid_results")
    else:
        mean = sum(results) / len(results)
        std = (sum((x - mean) ** 2 for x in results) / len(results)) ** 0.5
        support_fraction = sum(1 for r in results if abs(r) >= 0.7) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
        else:
            first_failing_seed = seeds[results.index(next(r for r in results if abs(r) < 0.7))]
            print(f"RESULT: FALSIFIED counterexample=\"slope < 0.7\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_784f5baa.py", line 102, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_784f5baa.py", line 43, in run_trial
    formula = generate_formula(n)
               ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_784f5baa.py", line 22, in generate_formula
    left = generate_formula(random.randint(1, n-2))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_784f5baa.py", line 22, in generate_formula
    left = generate_formula(random.randint(1, n-2))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(1, 1)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13549
2	preregistration	ollama_remote	glm4:latest	0	0	9516
3	novelty	ollama_remote	glm4:latest	0	0	8285
4	novelty	ollama_remote	glm4:latest	0	0	9090
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15709
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14440
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11819
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11162
9	judge	ollama_remote	glm4:latest	0	0	11656

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105225 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f3906e3e40c2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f3906e3e40c2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f3906e3e40c2.tar.gz (if generated)